LABORATORY USER MANUAL

ADwin Pro II

Python driver

Connection, everyday operations, and standalone measurements

Sarth Mhatre

Engineering Physics
IIT Indore

Last updated: 30 September 2026

Device	One ADwin Pro II, configured as device 1
Inputs / outputs	Eight ADC channels and eight DAC channels
Process	Generic I/O process, process number 1
Module mapping	ADC module 2; DAC module 3
Python interface	`ADwinPro2`, built on ADwin and QCoDeS

This manual documents the public operations needed for normal use. Internal helpers, register handling, and GUI program-writing conventions are intentionally outside its scope. GUI installation and controls are documented in the repository README.

1 Prepare the device and software

Read this first. The Python driver works with the supplied generic ADbasic process. It is not a universal interface to an arbitrary process already running on the ADwin.

1.1 Hardware and vendor software

1. Install the vendor's ADwin software, Windows device drivers, processor boot files, and ADbasic compiler on the control computer.
2. Connect and power the Pro II using the lab's configured connection. Use the vendor's tools to confirm that it is reachable as **device 1**.
3. Confirm that the process matches the chassis: ADC module 2, DAC module 3, and channels 1–8. The driver converts values using a −10 to +10 V range.
4. Choose wiring and voltage limits appropriate for the modules and sample. The driver's accepted range does not establish the sample's allowable voltage.

1.2 Prepare the process binary

Open the project's `pro2_io.bas` file in ADbasic. Select the processor target for your device, compile for **process 1**, resolve compiler diagnostics, and save the resulting binary. The Python constructor needs the compiled binary path, not the source-file path.

The supplied source references `ADC_DATA`, but the reviewed copy contains no explicit declaration for that buffer. If the compiler reports it as undeclared, resolve the acquisition-buffer declaration before compilation. The source has not been compiled or validated on hardware as part of the GUI software checks.

1.3 Python environment

Create the virtual environment once and install the project requirements. After that, select this environment in your IDE or notebook.

```
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

You do not need to add the driver folder to `sys.path` when running from the repository folder and importing it as a package. The folder `ADwin_Pro2_Generic_Driver` must be inside the project folder.

The vendor software is installed separately. Installing the Python ADwin package does not supply the entire device setup or ADbasic compiler.

Import compatibility The working driver uses `from qcodes import validators as vals`. The old import through `qcodes.utils` fails with QCoDeS 0.61.0. This is an import-location change; it does not alter the hardware command logic.

2 Open a session

Run this in a script or notebook using the project's virtual environment. The import is just a normal Python package import—no `sys.path` edits are needed when the notebook or script is run from the project folder. Replace the compiled-process path with the real path on your computer. Importing the class does not connect to the device; constructing an instance boots the device, loads the process, and starts it.

```
from ADwin_Pro2_Generic_Driver.adwin_pro2 import ADwinPro2

# Point this to the compiled ADbasic process file.
PROCESS_FILE = r"C:\path\to\compiled\pro2_io.TB1"

adwin = ADwinPro2(
    "adwin",
    device_number=1,
    process_file=PROCESS_FILE,
)

# Start from a known set of output requests.
adwin.set_all_dacs([0.0] * 8)
print("Process status:", adwin.process_status())
```

Argument	Meaning
`name`	QCoDeS instrument name; use one unique name per active instance.
`device_number`	Defaults to 1. This manual covers the single-device setup.
`boot_file`	Optional processor boot file. If omitted, the driver uses its configured default boot file. Verify that it matches the processor.
`process_file`	Required path to the compiled generic process binary.
`timeout`	Command-completion timeout in seconds; default 5.0.
`poll_interval`	Host completion-check interval in seconds; default 0.001. It does not set the ADC sample rate.

Why initialize all DACs? The process initializes physical outputs near zero, but the reviewed source does not explicitly initialize all stored output setpoints. Single-channel updates reapply the stored values for all channels. Use `set_all_dacs([0.0] * 8)` before individual-channel operations.

Device 1 only The current wrapper creates the vendor object with its default device selection (1); its `device_number` argument is stored but not forwarded. Leave it at 1. Selecting another device requires a driver change.

Keep the same instance for the session. Reconstructing it repeats boot and process startup; it is not a harmless connection check. A running process normally reports status 1; status 0 indicates that it is stopped.

3 Read inputs and set outputs

Channel names run from `adc1` to `adc8` and `dac1` to `dac8`. Values are expressed in volts. ADC parameters are read-only.

3.1 Read one input or all inputs

```
voltage = adwin.adc2.get()
print(f"ADC 2: {voltage:.6f} V")

readings = adwin.get_all_adcs()
for channel, value in enumerate(readings, start=1):
    print(f"ADC {channel}: {value:.6f} V")
```

`get_all_adcs()` returns an eight-element list in channel order: index 0 is ADC 1, index 1 is ADC 2, and so on. Use it when you need all channels from one acquisition request. Repeated individual `adcN.get()` calls make separate requests; they are not one shared sample.

3.2 Set one output or all outputs

```
# Apply a small voltage chosen for the connected experiment.
adwin.dac1.set(0.10)

# Exactly eight values, in DAC 1 through DAC 8 order.
adwin.set_all_dacs([0.10, 0.20, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])

# Replace every stored setpoint with zero and request an update.
adwin.set_all_dacs([0.0] * 8)
```

Each setpoint must lie between -10 and $+10$ V. `set_all_dacs` requires exactly eight entries and updates all channels in one command. This does not promise electrically simultaneous output changes. Use the all-channel call for a complete known output state, and the individual call for a later adjustment.

3.3 Read stored output setpoints

```
requested = adwin.dac1.get()
all_requested = adwin.get_all_dacs()
```

These values come from the stored DAC settings. They are **not a voltage measurement at the output connector**. A zero override or emergency state can hold the physical output near zero while these getters still report an earlier setpoint. Use an appropriate measurement to verify the physical output.

The conversion is quantized, so a stored value may differ slightly from the requested voltage. The process uses midscale code 32768 for nominal zero; do not interpret a small conversion offset as a calibrated output measurement.

4 Zeroing, emergency recovery, and closing

4.1 Temporarily hold one channel at zero

```
adwin.dac2.set(0.20)
adwin.zero_dac(2)       # Hold DAC 2 near zero; retain its setpoint.
adwin.restart_dac(2)    # Release the override; restore its setpoint.
```

`zero_dac(channel)` applies a zero override to a channel numbered 1–8. It does not discard the stored setpoint. `restart_dac(channel)` releases the override and may restore the previous voltage. Both calls trigger the process's output-update operation, which reapplies the other stored setpoints as well. Setting a channel while its zero override is active updates the stored value but does not release the override.

4.2 Software emergency state

```
adwin.emergency_stop()
active = adwin.emergency.get()  # 1 means requested emergency state.
```

While the generic process is running and responsive, it handles the emergency request by holding all DACs near zero. The request depends on software and communication; it is not a hardware interlock or confirmation of the measured output. Normal measurement commands may abort or time out in this state.

After addressing the reason for the stop, restore a known state:

```
# No other code should be sending commands during recovery.
adwin.emergency_reset()
adwin.set_all_dacs([0.0] * 8)
```

Reset clears the emergency/command state, but does not erase stored voltages or per-channel zero overrides. The following all-channel zero command replaces the old setpoints before later output operations. Do not resume another worker between these steps. Use `restart_dac(channel)` only for channels whose zero overrides you intentionally want to release.

4.3 End a normal session

```
try:
    adwin.set_all_dacs([0.0] * 8)
finally:
    adwin.close()
```

Complete measurement work before running this cleanup. `close()` stops the generic process if running and releases the QCoDeS instrument. It does not itself zero DACs; stopping a process can leave outputs at their last values. If zeroing raises an error, do not treat cleanup as proof of zero voltage.

For a normal pause, `stop_process()` stops process 1 and `start_process()` starts it again. ADC/DAC commands require it to be running. Restarting executes the process initialization again, including its output and state resets; it is not a continuation of an interrupted command.

5 Write a standalone measurement

This example assumes the `adwin` object from the previous section is already connected. It sets DAC 1, waits for the signal to settle, reads ADC 2, and saves the results to a CSV file in the current working folder. Choose safe voltages and a suitable settling time for your circuit.

```
import csv
import time

output = "standalone_measurement.csv"
voltages = [0.0, 0.05, 0.10, 0.15, 0.20]
settle_seconds = 0.10

try:
    # Opening with "w" replaces an existing file of this name.
    with open(output, "w", newline="", encoding="utf-8") as stream:
        writer = csv.writer(stream)
        writer.writerow(["elapsed_s", "dac1_requested_V", "adc2_V"])
        start = time.monotonic()

        for requested in voltages:
            adwin.dac1.set(requested)
            time.sleep(settle_seconds)
            measured = adwin.adc2.get()
            elapsed = time.monotonic() - start

            writer.writerow([elapsed, requested, measured])
            stream.flush()
            print(f"DAC 1 {requested:.3f} V; ADC 2 {measured:.6f} V")
finally:
    try:
        adwin.set_all_dacs([0.0] * 8)
    finally:
        adwin.close()
```

Order matters. Set the voltage, wait for the circuit to settle, then read the input. A completed DAC command does not mean the external circuit has finished settling.

Python controls the timing here. Each measurement includes Python sleep and communication delays, so this is for sequential measurements—not precise real-time sampling. Use an ADbasic acquisition process when timing must be controlled on the ADwin itself.

Use one command owner. Run standalone operations sequentially through one instance. Do not have another thread, notebook cell, or application send commands at the same time. The driver's internal locking does not make every compound measurement atomic across independent callers.

On interruption. Normal Python exceptions and keyboard interruption enter the `finally` block. A terminated kernel, disconnected device, or power failure can prevent cleanup. If the process is already in emergency mode, normal zeroing may fail; use the recovery procedure in Section 4.

6 Quick reference and troubleshooting

Operation	Use
`adwin.adc2.get()`	Read ADC 2 in volts.
`adwin.get_all_adcs()`	Read eight inputs; list in channel order.
`adwin.dac1.set(0.1)`	Set DAC 1 to a requested voltage.
`adwin.set_all_dacs(values)`	Apply eight ordered setpoints.
`adwin.dac1.get()`	Get stored DAC 1 setpoint, not physical readback.
`adwin.get_all_dacs()`	Get all eight stored setpoints.
`adwin.zero_dac(2)`	Hold DAC 2 near zero, retaining its setpoint.
`adwin.restart_dac(2)`	Release DAC 2 zero override.
`adwin.emergency_stop()`	Request all-channel emergency zeroing.
`adwin.emergency.get()`	Read the emergency-request flag.
`adwin.emergency_reset()`	Clear emergency and command state.
`adwin.process_status()`	Check generic process status.
`adwin.stop_process()`	Stop process 1; does not guarantee zero output.
`adwin.start_process()`	Start process 1; initialization runs again.
`adwin.close()`	Stop the process and release the instrument.

If something does not work

Driver import fails
Check the driver folder path, selected environment, and validators import. Restart the notebook kernel after editing the driver.

Boot or connection fails
Confirm vendor software installation, device 1 availability, the processor's boot file, and the compiled process-binary path. A source file is not a binary.

Read or write times out
Check that the compatible generic process is running on process 1, the module mapping matches, and emergency state is cleared. Raising the timeout does not repair a stopped or incompatible process.

An output stays near zero
Check emergency state and that channel's zero override before issuing further commands. Release an override only after reviewing the stored setpoint.

Duplicate instrument name
Reuse the existing instance or close it before constructing another. Do not reboot the device merely to refresh a notebook display.

Scope of verification: driver imports were checked with QCoDeS 0.61.0. No device was operated and no ADbasic hardware binary was validated while preparing this documentation.